Rolling the context: a drift-aware memory gate for long-running
LLM conversations (system study)

RLM rolling-window benchmark project
`rlm-gate-benchmark`

Hermes research agent
`hermes@local`

August 15, 2026

Abstract

Long-running LLM conversations pay attention to the entire history on every turn; on a consumer GPU the generation rate roughly halves from 8 K to 196 K context ($24.4 \rightarrow 8.7$ tokens/s). We study a simple remedy: keep a small *working window* and roll it forward, compacting the oldest portion into a summary at fixed context-fill gates (30%/60%/90%) and re-injecting survivors from a durable memory (the "RLM context gate" (PrimeIntellect, 2026)). On a 245,760-token conversation the rolling window at 32,768 tokens runs **1.67**× faster than the full context (12,148 s vs 20,242 s). Accuracy is not automatic: a bare rolling window recovers 75% of 12 facts. We introduce a *drift taxonomy* (absent / unbound / collision / paraphrased / stale / hallucinated) and a *forget-landscape reminder* that codifies each fact as a model-chosen `key|emoji|value` pair and re-injects it once per forget-quarter it has aged. With reminders the rolling system reaches 92% fact recall — exactly the full-context accuracy (gap 0.000) — with 3 of 4 misses eliminated. The residual miss is a same-shape value *collision*, a distinct failure mode we leave as the primary open problem. The study is a mechanism demonstration on one model/GPU/conversation; Section 7 specifies the experimental program required to turn it into a peer-reviewable claim.

Keywords: long-context LLM serving, context management, rolling window, memory, forgetfulness, drift analysis, reproducible systems research

1 Introduction

The dominant per-turn cost of a long conversation is re-reading history. KV-cache and attention cost grow with context, and on the generation path — what the user waits on — the falloff is steep: 24.4 tokens/s at 8 K context to 8.7 tokens/s at 196 K on an 8 GB GPU with a 35B MoE model (Section 3). The standard answer, *summarise the old history*, is the foundation of "memory" systems, but the literature rarely reports the accuracy cost of the roll itself, or the precise shape of the failures.

We contribute a benchmark and a mechanism:

- **A measurement** of the speed/accuracy trade-off of rolling windows, including the counter-intuitive result that the sweet spot is not the smallest window (too many summaries), and that a drift-aware memory closes the accuracy gap to full context *exactly* (Section 5).

- **A drift taxonomy** that says *how* a rolling context fails — not just whether it does — and a **forget-landscape reminder** whose re-injection multiplicity grows with a fact's age quarter (Section 6).
- **An explicit roadmap** for turning this system study into a paper (Section 7).

2 Related work

Context compression and memory. Summarising-and-dropping history is the classic pattern (Wu et al., 2022; Xu et al., 2023). More recent agent memories (Park et al., 2023; Packer et al., 2023) interleave user turns with "memory" turns written by the system. Our gate is a serving-layer adaptation of the *recursive language model* (RLM) abstraction (PrimeIntellect, 2026), which treats context as a variable inside a persistent REPL and keeps durable memory state in a continual harness. We differ in that our gate is *thresholded by context fill* (30/60/90%) and *drift-aware*: re-injection multiplicity is proportional to how far a fact has aged, not merely whether it exists.

Long-context attention. Sparse attention (Child et al., 2019; Beltagy et al., 2020), KV eviction (Zhang et al., 2024; Ge et al., 2024), and sliding windows (Beltagy et al., 2020) attack the same wall-clock problem inside the model. We benchmark a *serving-layer* strategy that composes with any of these.

Benchmarking LLM serving. Throughput-at-context studies (Bao et al., 2024) focus on batching and scheduling; we focus on the single-conversation latency curve as a function of context.

3 Setup and speed sweep

Platform. A single 8 GB GPU; llama.cpp `llama-server` on `localhost:8082` with `-metrics`, `n_ctx` = 250,112, KV cache in `q8_0`, MoE FFN experts on CPU. Model: Qwen3.6-35B-A3B-UD-IQ2_M (35B MoE, IQ2_M quant, 11.5 GB). Temperature 0.0; probes generate 16 tokens; each point is the best of two runs.

Sweep. Context sizes 8,192 . . . 196,608:

Context (tokens)	Prefill t/s	Generation t/s	GPU util %
8,192	**308.3**	24.4	79
16,384	298.1	21.6	77
32,768	298.1	22.3	81
65,536	289.2	17.9	83.5
131,072	253.1	12.9	83
196,608	215.9	8.7	81

Prefill degrades gracefully; generation does not. This is the budget a rolling window recovers. (*Sweep cap:* the 245,760 point is unreachable — prompt construction overshoots `n_ctx` by $\approx 8\%$ and the server returns HTTP 400; the baseline caps at 196,608.)

4 Rolling window + RLM gate

The gate. The conversation runs in a fixed window. When context fill crosses 30%/60%/90%, the oldest portion is compacted into a summary — a *gate deposit* — which is re-injected into the next window; the gate re-arms at the 90% compaction. Summaries are generated from the warm KV cache (`summary_from_cache=True`), the realistic serving path.

Wall time. Same 245,760-token conversation, full context vs rolling:

Mode	Window	Wall (s)	Combined t/s	Deposits	Speedup
Full context	245,760	20,242	12.14	—	1.00×
Rolling+RLM	8,192	12,976	18.94	100	1.56×
Rolling+RLM	16,384	12,258	20.05	49	1.65×
Rolling+RLM	**32,768**	**12,148**	**20.23**	**24**	**1.67**×
Rolling+RLM	65,536	12,466	19.71	12	1.62×

The sweet spot is 32,768: the 8 K window summarises too often (100 deposits) and pays for it; the 65 K window saves little further. `gate_deposits` is a diagnostic ledger of memory work.

5 Accuracy and the drift taxonomy

Eval. A synthetic 12-fact conversation (6,012 tokens) describing a system deployment, with facts of controlled shape (`number:unit`, `word`, `abbrev`, `letter_number:context`, ...), ten seeded early (old) and two late (recent). The model is a *thinking* model, so the eval uses `/chat/completions` with `enable_thinking: false` and reads `message.content`. Each fact is probed once; an answer matches the seeded value under normalisation.

	Full context	Rolling + RLM (reminders)
Accuracy	92%	**92%**
Accuracy gap	—	**0.000**
Old-fact accuracy	—	90%
Recent-fact accuracy	—	100%
Tier recall (0.3/0.6/0.9)	—	0.167 / 0.25 / 0.167

6 Drift-aware reminders

Taxonomy. Every miss is classified by **mode** (absent / unbound / collision / paraphrased / stale / hallucinated) and **shape** (answer type). Without reminders the rolling window misses 3/12 facts (75%):

Fact	Expected	Got	Shape	Mode
`context`	250K	8GB	number:unit	collision
`gate`	ninety	90	word	unbound
`memory`	RLM	S3	abbrev	collision

Two *collisions* (the value merged with the wrong attribute) and one *unbound* value (`gate` $\rightarrow$ 90 — the value survived, its attribute binding did not; the gate's own 90% threshold collides with "ninety"). Recall by forget-quarter: Q1 = 0.667, Q2 = 0.333 — the middle of the conversation, not the very oldest, is where facts vanish.

Reminders. Each fact is codified as `key|emoji|value`, with the emoji *chosen by the model per run* (no hardcoding; the legend in `eval_after_summary.json` — e.g. `context|book`, `gate|traf- fic_light`, `memory|abacus`, `rolling|arrows_ccw`). A fact's *age quarter* is its position mapped to $\{0, 1, 2, 3\}$. Re-injection multiplicity:

$$\text{multiplicity} = 1 + \text{age\_quarters}.$$

Both the RLM memory and the working context carry the pair (shared responsibility). *Before / after:*

Condition	Misses	Rolling accuracy	Gap vs full context
Rolling, no reminders	3 / 12	75%	+0.167
Rolling + reminders	**1 / 12**	**92%**	**0.000**

The residual miss is a `number:unit` *collision* (`rolling` 64K vs `context` 250K): two same-shape facts merged. This is the open problem – not forgetfulness but wrong-value binding.

7 Path to a paper

This is the section a follow-up agent can pick up and work from. The study above is a single-condition mechanism demonstration. The following items, in order, convert it into a peer-reviewable claim. Each is a concrete, verifiable task.

R1. Multi-model / multi-seed sweep (highest priority). Reproduce the sweep and eval on ≥ 3 models (e.g. Qwen3-8B, Llama-3.2-3B, Llama-3.1-8B) $\times \geq 5$ seeds each, at 2–3 GPU sizes. Report mean $\pm$ CI for wall time, accuracy, gap, and tier recall; test the gap with a paired test (e.g. Wilcoxon) across conversations. *Exit criterion:* a results table with error bars and a p-value per row.

R2. Real long-conversation eval. Replace the synthetic 12-fact conversation with a real long-running task (e.g. a multi-turn code-debugging or agentic web task) and a fact probe derived from the ground truth. *Exit criterion:* the eval answers the question "do facts from a real conversation survive a roll?"

R3. Baselines. Compare against (a) plain truncation (drop-oldest, no memory), (b) a naive summarise-all memory (no gate, no drift), (c) RAG (fact retrieval on every turn), and (d) MemGPT-style interleaved memory turns. *Exit criterion:* a table with at least four methods $\times$ the accuracy metrics of Section 6.

R4. Ablations. (a) reminders off / on; (b) multiplicity $= 1$ (flat) vs 1+age; (c) hardcoded vs model-chosen emojis (measure stability of the legend across runs); (d) gate thresholds (20/50/80 vs 30/60/90); (e) summary-from-cache on/off. *Exit criterion:* one paragraph per ablation with a numbers table.

R5. Attack the collision mode. Design reminders that disambiguate same-shape facts (attribute-qualified pairs, e.g. `context_size=250K` vs `rolling_window=64K` separated from numeric-only tokens), and measure the collision rate as a dedicated metric. *Exit criterion:* collision count reported per condition; a proposed fix tested.

R6. Config and ethical/repro section. Full server flags, quantisation, KV dtype, temperature, and generation seed in an appendix; a statement on synthetic-data limitations; a data-availability note. *Exit criterion:* a reviewer can reproduce the headline numbers from the repo.

R7. YaRN comparison (scoped as future work). The locked serving config uses a 30 K native window. Compare YaRN options (35B @ 30 K + YaRN 4×; 35B @ 128 K + YaRN 4×) against the rolling gate on the same conversation. *Exit criterion:* a wall-time vs accuracy trade-off plot including YaRN curves.

Venue. Systems/workshop format (e.g. LLM-Serving workshop, memory-workshop) fits a single-model study; a full paper should target a main conference only after R1–R3.

8 Conclusions

Rolling the context with a drift-aware gate is, in this benchmark, strictly better on both axes: **1.67× faster and within 0.000 of full-context accuracy.** The drift taxonomy turns "it forgets" into "it loses attribute bindings (unbound) and merges same-shape values (collision)", and the forget-landscape reminder (multiplicity $= 1 + \text{age}$) eliminates both observed miss classes but one collision. Section 7 states precisely what must be done next.

References

PrimeIntellect. Prime agent: A self-improving RLM agent (software, MIT). *https://github.com/PrimeIntellect-ai/prime-agent*; Continual Harness, *arXiv:2605.09998*, 2026.

J. Wu, L. Ouyang, D. Ziegler, N. Stiennon, R. Lowe, J. Leike, and P. Christiano. Recursively summarizing books with human feedback. *arXiv preprint*, 2022.

X. Xu, Z. Gou, W. Wu, K. Niu, H. Wei, H. Yang, Y. Piao, B. Wang, and Z. Chen. Notion: A large-scale training dataset for long-text summarization. 2023.

J. S. Park, J. O'Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein. Generative agents: Interactive simulacra of human behavior. *Proceedings of UIST*, 2023.

C. Packer, S. Wooders, K. Lin, V. Fang, S. G. Patil, I. Stoica, and J. E. Gonzalez. MemGPT: Towards LLMs as operating systems. *arXiv preprint*, 2023.

R. Child, S. Gray, A. Radford, and I. Sutskever. Generating long sequences with sparse transformers. *arXiv preprint*, 2019.

I. Beltagy, M. E. Peters, and A. Cohan. Longformer: The long-document transformer. *arXiv preprint*, 2020.

Z. Zhang, Y. Sheng, T. Zhou, T. Chen, L. Zheng, R. Cai, Z. Song, Y. Wang, Z. Chen, H. Zhang, et al. H_2O: Heavy-hitter oracle for efficient generative inference of large language models. *NeurIPS*, 2024.

S. Ge, Y. Zhang, L. Liu, M. Zhang, J. Han, and J. Gao. Model tells you what to discard: Adaptive KV cache compression for long-context LLMs. *ICLR*, 2024.

Y. Bao et al. RLBench: Benchmarking the serving throughput of large language models. 2024.

A Reproduction

All code, figures, tests, and results live in the repository (`rlm-gate-benchmark`). Commands: `python -m src.sweep`, `python -m src.rolling_window`, `python -m src.analyze`, `python scripts/check_figures.py`. Unit tests (21) cover the drift taxonomy, shape classification, quarter math, and reminder multiplicity; they run without pytest.

B Drift profile

`results/drift_profile.json` contains the per-condition miss ledger used in Section 6: modes, shapes, per-quarter recall, and individual miss records.